

P395: Guide 7: Partial Differential Equations

1. Start up a new notebook and title it 'Guide7_your_name'. Enter

```
import matplotlib.pyplot as plt
import numpy as np
```

Introduction

This week we will look at solving partial differential equations (PDEs) that give solutions that depend on several variables. Common physics examples include the 1D wave equation, where the amplitude of the wave depends on both position x and time t , or Laplace's equation for the electrostatic potential which depends on position (e.g., x , y , and z).

The topic of partial differential equations and how to solve them both analytically and numerically is vast. By doing several applications, you will learn a few of the basic techniques for solving different classes of PDEs.

But first, let's go over some notation. For the PDEs that follow, we will have functions that are just functions of two variables, either time and position, $u(t, x)$, or possibly two positions, $u(x, y)$. To simplify notation, we will write derivatives as

$$\frac{\partial u}{\partial t} = u_t$$
$$\frac{\partial^2 u}{\partial x \partial y} = u_{xy}$$

So for example, Laplace's equation ($\nabla^2 u = 0$) in 2D cartesian coordinates would be written as

$$u_{xx} + u_{yy} = 0$$

Categories of PDEs

The three types of equations we will consider in the problems will be of the form

$$\begin{aligned} u_{xx} + u_{yy} &= 0 && \text{(Laplace's equation)} \\ u_t &= u_{xx} && \text{(Heat/Diffusion equation)} \\ u_{tt} &= u_{xx} && \text{(Wave equation)} \end{aligned}$$

In general, a partial differential equation in two variables (here let (x, y) represent 2 spatial dimensions or one space and one time) is of the general form

$$au_{xx} + bu_{xy} + cu_{yy} + du_x + eu_y + fu + g = 0$$

and leads to the general classification into 3 types,

$$\begin{aligned} b^2 - 4ac &> 0 && \text{(hyperbolic, e.g., the wave equation)} \\ b^2 - 4ac &= 0 && \text{(parabolic, e.g., the heat/diffusion equation)} \\ b^2 - 4ac &< 0 && \text{(elliptic, e.g., Laplace's equation)} \end{aligned}$$

Task 1: Solving Laplace's equation

In electrostatics, in regions that are free from any charge density, the electrostatic potential satisfies Laplace's equation. For a 2D Cartesian geometry, Laplace's equation for the potential $V(x, y)$ is

$$V_{xx} + V_{yy} = 0$$

with time-independent boundary conditions (and potentially at certain points within the region) where the potential is held at fixed values. To solve such an equation numerically, one can start by discretizing space into a uniformly spaced grid in x, y with spacing h in each direction. The value of the potential at a grid point is $V(x_i, y_j) = V_{i,j}$. The finite-difference formula for the 2nd derivative at grid point (i, j) is (see the Wikipedia entry on finite differences),

$$V_{xx} = \frac{V_{i-1,j} - 2V_{i,j} + V_{i+1,j}}{h^2}$$

and similarly for V_{yy} except now the displacements are along the column axis j .

1. **Analysis:** **Show** that using the finite-difference formulas for V_{xx} and V_{yy} in Laplace's equation leads to the following equation for the potential at grid point (i, j) ,

$$V_{i,j} = \frac{1}{4}(V_{i-1,j} + V_{i+1,j} + V_{i,j-1} + V_{i,j+1})$$

which is just the average of the potential at the surrounding sites.

This result gives an iterative scheme (known as the Jacobi method) for finding the potential. Given some value $V_{i,j}^k$ for the potential at the k^{th} iteration, the above equation gives an update rule for V at the $(k+1)$ th iteration,

$$V_{i,j}^{k+1} = \frac{1}{4}(V_{i-1,j}^k + V_{i+1,j}^k + V_{i,j-1}^k + V_{i,j+1}^k) \quad (\text{Jacobi method})$$

Given some initial guess for the V 's and boundary conditions on V (and potentially some interior locations), this equation can be iterated until convergence for the grid points on the interior.

The Jacobi method can be slow to converge. A slightly faster method (the Gauss-Seidel method) is based on realizing that some of the grid points in the above equation have already been updated, and those updated values can be used instead. Its update formula is

$$V_{i,j}^{k+1} = \frac{1}{4}(V_{i-1,j}^{k+1} + V_{i+1,j}^k + V_{i,j-1}^{k+1} + V_{i,j+1}^k) \quad (\text{Gauss-Seidel method})$$

It uses the updated values at the prior row and column positions instead of their old values.

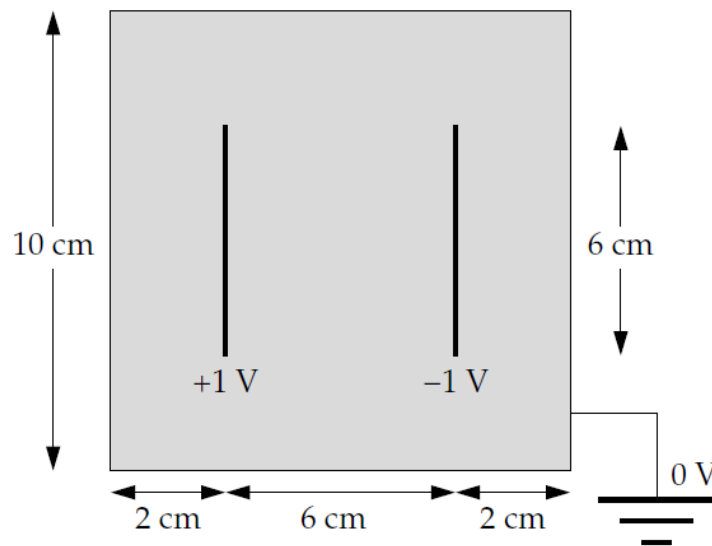
Potential with fixed boundaries

Solve Laplace's equation for the two-dimensional problem of a square box 1 m on each side, with boundary conditions of $V = 1$ volt along the bottom and right walls and $V = 0$ volts along the top and left walls. Use a grid of spacing $h = 1$ cm, giving 101 grid points along each direction.

2. Use the Jacobi update formula to calculate the electric potential until it changes by no more than $\delta = 10^{-6}$ Volts at any grid point on any step. Start with an initial guess for the interior grid points at a value of 0 V. How many iterations does it take?
3. Make a heatmap of the final solution using `plt.imshow`. I recommend setting the keyword `interpolation='None'`.
4. Repeat (2) and (3) except now use the Gauss-Seidel update formula. How many iterations does it take now to get the same tolerance?

Potential of a parallel-plate capacitor

A slightly more involved problem is to find the potential in the vicinity of a parallel-plate capacitor within a grounded metal box (see figure below). All boundaries are held at 0 V, and the capacitor has metal plates held at $V = +1$ Volt and $V = -1$ Volt, respectively.



5. Use the Jacobi method to find the potential everywhere inside the box on a grid of 101×101 points. Keep the grid points along the 2 edges of the capacitor fixed at +1 V and -1 V respectively and all the boundaries fixed at 0 V. Iterate until the incremental change is less than $1e-6$ V for any point within the region. How many iterations did it take to converge?
6. Make a heat map of the resulting potential.
7. **Analysis:** Comment on if the solution differs from what you learned about for an infinite parallel-plate capacitor.

Task 2: Solving the heat/diffusion equation

The heat equation for the time-dependent temperature profile $u(t, x)$ in a 1D system of length L is

$$u_t = \alpha u_{xx}$$

where α is the thermal diffusivity of the material. It is subject to the initial conditions $u(0, x) = u_0(x)$ and (potentially time-dependent) conditions on the boundaries $u(t, 0) = T_0(t)$ and $u(t, L) = T_L(t)$. As we did for Laplace's equation, this equation can be solved numerically by discretizing space into a

uniformly spaced grid of $N+1$ points separated by h , giving $x_i = ih$, and discretizing time into steps of size Δt , giving $t_k = k\Delta t$. Let u_i^k be the value of the function at grid point (t_k, x_i) .

Using the forward Euler method for time and the central-difference method for the spatial derivatives, one arrives at the **FTCS** (forward-time central-space) equation for the heat equation,

$$\frac{u_i^{k+1} - u_i^k}{\Delta t} = \alpha \frac{u_{i-1}^k - 2u_i^k + u_{i+1}^k}{h^2}$$

This leads to the following update formula for the value at position i at the $(k+1)$ th time step,

$$u_i^{k+1} = u_i^k + \frac{\alpha \Delta t}{h^2} (u_{i-1}^k - 2u_i^k + u_{i+1}^k) \quad (\text{FTCS method})$$

The boundary conditions translate to $u_0^k = T_0(t_k)$, $u_{N+1}^k = T_L(t_k)$, and $u_i^0 = u_0(x_i)$. Starting with an initial temperature profile at $t=0$ and with the boundary conditions, this equation gives an iterative procedure for calculating the temperature at the interior points for all times. However, there is one caveat. It is found that one needs $\alpha \Delta t / h^2 < 1/2$ for this iterative procedure to be stable. Or put another way, the stability condition for given α and spacing h requires that

$$\Delta t < h^2 / (2\alpha) \quad (\text{Stability condition on FTCS})$$

Thermal diffusion through Earth (variation on problem by Mark Newman)

Here you will calculate how the temperature varies as a function of depth below the Earth's surface as the season changes. We will calculate the temperature from the surface ($x=0$) to a depth $x=L$ below the surface. The boundary condition at the surface ($x=0$) has a time-dependent temperature due to solar heating,

$$T_0(t) = A + B \sin\left(\frac{2\pi t}{\tau} + \phi\right)$$

where $\tau = 365$ days, $t=0$ corresponds to January 1, constants A and B are the average temperature and the maximum temperature change (from average), which depend on the location on the earth, and ϕ sets the phase. The boundary condition at a depth $x=L=40$ m has the temperature constant at

$$T_L(t) = A + m_T L$$

where $m_T = 30^\circ\text{C}/\text{km}$. The thermal diffusivity through the Earth's crust is on average around $\alpha = 0.1$ m^2/day .

Let's consider our location to be in Canada's arctic, with $A = -12^\circ\text{C}$ and $B = 20^\circ\text{C}$, and $\phi = -1.9$ radian gives the lowest temperature in January with a peak in July.

1. **Plot** surface ground temperature $T_0(t)$ over a whole period of 365 days. What are the surface temperatures on the coldest and hottest day?
2. **Solve** the heat equation using the FTCS method for the temperature profile (from $x=0$ to $x=L=40$ m) over a 10-year period. Choose a reasonable number of spatial points (e.g., 101) and then choose a Δt that satisfies the stability condition above. Start with a guess for the initial profile of $T(0, x) = A$, except at the boundaries where $T(0, 0) = A + B \sin(\phi)$ and $T(0, L) = A + m_T L$ (of course, this is not what the profile looks like, but you have to start with something). Instead

of saving all your solutions in an array, **store only** the daily temperature profiles of the 10th year in an array. **Hint:** using the FTCS method, you can just keep overwriting your solution for $u(x,t)$ as you iterate. In the last year, store the daily solutions in an array, so that you have an array of 365 temperature profiles at the end).

Note: As you iterate forward in time, you will keep the temperature at the right boundary ($x=L$) fixed, but the surface temperature will vary in time as given by the above equation for $T_0(t_k)$. It will take a few years for the temperature profiles to stabilize and get rid of the initial transients due to the initial guess.

3. Plot four temperature profiles from the last year that are separated by 3-month intervals.
4. **Analysis:** What are some interesting features that you note?
5. Using the profiles from the last year, find both the maximum and minimum yearly temperature at each depth x . Plot your results for the minimum and maximum temperature as a function of depth. These should 'envelope' your plots in (3).
6. **Analysis:** From the graph in (5), can you state at what depth the permafrost begins? Permafrost is the depth which never gets above 0 degrees Celsius over an entire year.

Task 3: Solving the wave equation

The time variation of the amplitude of a wave at position x along a clamped string of length L satisfies the PDE

$$u_{tt} = c^2 u_{xx}$$

where the speed of the wave is c . It is subject to initial condition $u(0, x) = u_0(x)$ and boundary conditions $u(t, 0) = 0$ and $u(t, L) = 0$. To solve this numerically, we will discretize space and time as before, but now will use a central-difference formula for both space and time, yielding

$$\frac{u_i^{k-1} - 2u_i^k + u_i^{k+1}}{\Delta t^2} = c^2 \frac{u_{i-1}^k - 2u_i^k + u_{i+1}^k}{h^2}$$

This results in the following explicit update formula

$$u_i^{k+1} = 2u_i^k - u_i^{k-1} + \frac{c^2 \Delta t^2}{h^2} (u_{i-1}^k - 2u_i^k + u_{i+1}^k)$$

This is like what we had for the heat equation, except now we need the solutions at the two previous time steps, k and $(k-1)$. Thus, we need to keep track of two profiles (the current and previous) to make the update. To start the iterative procedure, we need values for the amplitude of our wave at u_i^0 and u_i^1 . We will hold the string in some nonequilibrium shape $g(x)$ (i.e., like you would when you pluck it) so that $u_i^1 = u_i^0 = g(x_i)$, after which point the string is let go and the wave develops. With u_i^0 and u_i^1 specified, the update formula can be used to evolve the waveform in time.

Wave on a string

Consider the G string of a guitar. It has a length $L = 0.65$ m and a wave speed of $c = 254.8$ m/s, which gives a fundamental frequency $f = 196$ Hz. The end points of the string are fixed such at $u(t, 0) = 0$ m and $u(t, L) = 0$ m. For the initial conditions, set the shape

$$u^0(x) = u^1(x) = g(x) = A \frac{x(L-x)}{L^2} \exp \left[-\frac{(x-d)^2}{2\sigma^2} \right]$$

Take $A = 0.025$ m, $d = 0.5$ m, and $\sigma = 0.3$ m. Set a reasonable number of spatial points (say 51) and a time step of $\Delta t = 1\text{e-}5$ s.

1. **Solve** the wave equation using the above update formula and calculate u_i^k from $t = 0$ to $t = 5$ s, **recording** the solution at a sampling rate of 10,000/s (i.e., you should get 50,000 waveforms).

Now you will animate your solutions. Turn on the interactive environment with `%matplotlib widget`.

2. Animate your wave solutions.

Now let's hear the sound made by your solution. To play the sound of a waveform in python:

3. Execute: `from IPython.display import Audio`.

Let's first hear what a pure tone of G at 196 Hz sounds like. This will generate a sound lasting for 5 s at a sampling rate of 10,000/s.

4. Run the following code in a cell

```
freq = 196.0      # frequency of G note
framerate = 10000

t = np.linspace(0, 5, framerate*5)

data = np.sin(2*np.pi*freq*t)

Audio(data, rate=framerate)
```

This will generate a little sound player that you can then press play on to hear the sound.

5. Pick a point near the middle of the string and **plot** its amplitude versus time from your solutions.
6. Using `Audio`, play the waveform from (5) using a framerate of 10,000/s.
7. **Analysis:** How does it sound compared to what you heard in (4) ? Does it sound the same or different? **Hint:** recall from 1st-year physics about the harmonics on a string.

Task 4: Wrap up

1. Please provide any feedback on questions that you found difficult to understand because of the wording.
2. Please provide any other comments about this notebook.